

CSE 490/590 – Project 1: Checkpoint 1

Member A: Faiyaz Rafee

Member B: Ilya Oleynik

Shared repository: <https://github.com/ioleynik/CSE490>

(Gotta remember to share repository with TAs)

16-bit, single-cycle, non-pipelined processor

Processor parts:

- Program counter
- Instruction memory
- Register file
- Arithmetic Logic Unit (ALU)
- Data memory
- Control logic
- Sign-extension logic
- Multiplexers
- Branch and jump address logic

Need to implement:

add, sub, sll, and, lw, sw, addi, beq, bne, jmp.

Design

We will use behavioral Verilog for individual components and structural Verilog to connect the components in the top-level processor.

Instruction flow:

1. The program counter gives the current instruction address
2. Instruction memory returns the 16-bit instruction.
3. The control unit decodes instruction then generates control signals.
4. Register file supplies the required register values.
5. ALU does arithmetic/logical operations/ memory-address calculations.
6. data memory is accessed for load and store instructions.
7. writeback multiplexer chooses the ALU result or loaded data for instructions that write a register.
8. Next-PC logic selects the sequential, branch, or jump address (+2 for next).

Register file will have 16 16-bit registers, \$s0 through \$s15.

Instruction memory and data memory are separated with 64 bytes+ per. Memory will be byte-addressed. We can potentially use arrays with 8-bit entries and then combine 2 adjacent bytes for the 16-bit words

requirement. Loads and stores will use big-endian byte order (the higher byte of a word is stored at the lower address)

Instruction Decoding layout

For R-type and I-type instructions:

- Bits [15:12]: opcode
- Bits [11:8]: rt/rd register
- Bits [7:4]: rs register
- Bits [3:0]: function code or immediate

For jump instructions:

- Bits [15:12]: opcode
- Bits [11:0]: signed jump offset

R-type instruction:

opcode				rt/rd				rs				function code			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

I-type instruction:

opcode				rt/rd				rs				immediate			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

J-type instruction:

opcode				address											
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Instructions cover 2 bytes each so normal execution advances the PC by 2

PC = the current instruction address:

- Sequential execution: next PC = PC + 2
- Taken branch: next PC = PC + 2 + (sign-extended 4-bit immediate shifted left by 1)
- Jump: next PC = PC + 2 + (sign-extended 12-bit offset shifted left by 1)

Work Division

Member A: Faiyaz Rafee

Member B: Ilya Oleynik

Member A: Datapath and Storage

main responsibilities:

- Implement the ALU.
- Implement the register file.
- Implement instruction memory.
- Implement data memory.
- Prepare memory initialization data.
- Test arithmetic, logical operations, register access, and memory access.
- Document component behavior and test results.

Initial tasks:

- Define register-file inputs, outputs, and write behavior.
- Plan byte-addressed instruction and data memories.
- Prepare basic ALU tests.

Member B: Control and Integration

Primary responsibilities:

- Implement the program counter.
- Implement instruction decoding and control signals.
- Implement sign extension and multiplexers.
- Implement branch and jump address calculations.
- Connect modules in the structural top-level processor.
- Prepare integrated instruction tests.
- Lead FPGA constraints, I/O mapping, and bitstream generation.
- Assemble the checkpoint and final reports using both members' contributions.

Initial tasks:

- Sketch the top-level component connections.
- Prepare a control-signal table.
- Define how the next instruction address is selected.
- Prepare basic PC tests.

We will both:

- Agree on initial interfaces before starting work
- Review each others modules and understand how they connect.
- Maintain the meeting log.
- Debug integrated behavior.
- Do FPGA testing.
- Review and work on reports

Schedule:

October 4: Checkpoint 1

- Review the initial processor design.
- Agree on the work division.
- Create the shared repository and provide TA access.
- Submit the checkpoint PDF.

October 5–11: Component Development

- Finalize module interfaces.
- Implement and test the main components.
- Integrate instruction fetch and R-type instruction execution.

October 12–18: Integration / Checkpoint 2

- Add immediate arithmetic, loads, and stores.
- Add conditional branches and jump.
- run integrated instruction tests.
- update architecture diagrams, code, logs, and progress records.
- Submit Checkpoint 2 by October 18 at 11:59 PM.

October 19–November 1: Final Part

- Complete processor testing.
- Fix any integration and synthesis issues.
- FPGA constraints and the hardware demonstration.
- Test the bitstream.
- Prepare and submit final source files and pdf by November 1 at 11:59 PM.

Meeting and Discussion Log:

Sept 30 - Met up and discussed project idea and created Github Repo - Both Faiyaz and Ilya

Oct 2 - Started work on the Project report and Verilog - Both Faiyaz and Ilya

Oct 4 - Finished up the Project Report, Repo setup, and basic Verilog code - Both Faiyaz and Ilya